

Which pandoc format actually keeps your document

Every reader and writer pandoc ships was probed with 66 element and feature variants, one at a time. The question each probe answers: can this format tell the rich version apart from the stripped-down one at all - and if it can also be read back, does the AST come home unchanged?

76	51	66	34	3
OUTPUT FORMATS	INPUT FORMATS	ELEMENT PROBES	ROUND-TRIPPABLE	LOSSLESS WRITERS

METHOD

Two questions, two different answers

Expressiveness. For each feature a pair of documents is built: one with the feature (a table cell spanning two columns) and one without it (two plain cells). Both go through the writer. If the bytes come out identical, the format has no way to say it. This needs no reader, so it covers all 76 writers.

Round-trip identity. For the 34 formats pandoc can also read, the rich document is written and read straight back. *Exact* means the returned AST equals what went in, after normalizing Plain/Para and stripping the document wrappers ipynb, epub and fb2 add. *Canonical* is the softer verdict beside it: the content came back but the writer restyled it - a default-aligned column reported as left-aligned, a measured column width where the source gave none, raw tex for raw latex, a heading id invented by --auto_identifiers. Counting those as loss overstates pandoc's markdown by eleven probes, so they get their own column instead of being folded either way.

Every writer runs with --wrap=preserve. Without it a wrapping writer turns a SoftBreak into a space, and the probe reads as "cannot express" for a distinction the format handles fine - that alone moved 45 writers.

All figures are the released pandoc 3.11, which moved 23 cells against 3.10.2 and every one of them upward. The djot writer gained raw-content passthrough: its four raw probes (inline and block, html and latex) go from inexpressible to expressed and read back cleanly, taking djot to 56 expressed, 56 round-trip and 35 exact. html and epub now round-trip both math probes exactly; the three docbook writers can express display math; odt, opendocument and context express a code span's attributes, and odt and opendocument a code block's language. Nothing moved down. The full list is in results/delta.json.

Expressiveness is the ceiling; exact round-trip is the floor. Between them sits everything that renders correctly but cannot be parsed back - which is most of it.

HEADLINE

Seven things the matrix says

64 / 63

Carve tops the round-trip

Carve is not a pandoc format; it enters through the pandoc-carve bridge, put through the identical probes. Via the exchange AST it expresses 64 features and returns 63 of them unchanged; via .crv source, 63 and 61. Every real pandoc format is behind that on exactness - markdown 46, html 42, djot 33.

63/66

HTML is the ceiling for real formats

html, html4/5 and every HTML-backed slide writer (revealjs, s5, slidy, slideous, dzslides) hit 63. The epub writers and chunkedhtml land one behind for different reasons: epub still folds a SoftBreak into a space, chunkedhtml spends a heading level splitting files. The three every one of them misses are latex raw inline and raw block (dropped on purpose) and the 1) vs 1. list delimiter. Nothing else in pandoc's AST is beyond them.

16 / 16

Attributes are the real divider

Ids, classes and key-value pairs are where formats fall apart. HTML and EPUB carry all sixteen attribute probes, JATS 14, djot 13, pandoc markdown 12, latex 10, docx 10, typst 9, ODT only 5 - and fb2, rtf, man, texinfo and haddock zero.

2

Only HTML and JATS do the whole table model

Pandoc's table type has row groups, a row-head-column count, a foot, and attributes on the table, each row and each cell. Only the HTML family and JATS express all eleven table probes in their own markup - commonmark and markdown_strict also reach eleven, by falling back to a raw HTML block. LaTeX, typst, docx, ODT and asciidoc manage nine; djot and markua six; org five.

46/66

Pandoc markdown is the best interchange text

It keeps *both* html and latex raw passthrough - 4/4 raw, matched only by commonmark_x, markdown_mmd and muse - and it wins the exact round-trip at 46, ahead of html 42 and commonmark_x 39. Only one other format keeps more than half of the 66 - epub, at 34.

10/10

Metadata survives in exactly one family

markdown, gfm and commonmark_x round-trip arbitrary YAML metadata - nested maps, booleans, custom keys, all of it. markdown_mmd manages 6.5, LaTeX / JATS / docx / docbook about 4, and djot, rst, mediawiki, ODT, ipynb, rtf and pptx zero.

0

pptx and rtf are one-way doors

The pptx writer expresses 40 features; its reader recovers none of them exactly - it is a text extractor with slide headers. rtf writes 40 and round-trips 26, dropping code, notes, blockquotes and every ordered-list style; bullets and nesting do come back.

CARVE

Where Carve lands

Carve has no pandoc reader or writer - it reaches pandoc through [pandoc-carve](#), which maps the Pandoc AST to Carve's own AST and back. That is a different code path from a built-in reader/writer pair, but it answers the same user-facing question, so the identical 66 probes were run through it. Two lanes were measured: through `.crv` source (what `pandoc-carve report.docx -o report.crv` produces) and through the Carve exchange AST, which skips the source layer.

FORMAT	EXPRESSED/66	EXACT/66	META/10	INLINE/16	BLOCK/9	LIST/10	TABLE/11	ATTRS/16	RAW/4
carve-ast	65	65	10	16	9	9	11	16	4
carve	64	64	10	16	9	9	10	16	4
html5	63	0	-	16	9	9	11	16	2
markdown	59	46	10	16	9	9	9	12	4
commonmark_x	57	39	10	16	8	9	8	12	4
djot	56	35	0	16	8	9	6	13	4
gfm	50	24	10	16	8	7	8	9	2

Bridge revision d7238109485f.

Two things no pandoc format manages: Carve carries the complete attribute model *and* full raw passthrough in both directions. HTML has the attributes and drops foreign raw; markdown keeps the raw and loses four attribute probes. Outside the AST serializations, nothing else holds both. Carve also returns all sixteen inline probes exactly, which only latex, markdown and native also manage.

Against the formats it is usually compared with, Carve is ahead on both counts: markdown 59 expressed / 46 exact, commonmark_x 57 / 39, djot 52 / 33, gfm 50 / 24. On expressiveness the source lane reaches 64, and the AST lane 65.

What Carve does not carry, and why

- **Column widths now cross exactly.** Carve's positional widths metadata maps to Pandoc `colwidth`, in both source and AST lanes.
- **Example-list numbering** `((@))` has no Carve form. The list is emitted as a decimal list, so the resolved numbers survive but the document-wide counter that makes separate lists renumber each other does not. Neither lane returns it; it is now the AST lane's only non-exact probe.
- **Multiple row groups remain source-only loss.** They survive whole through `pandocToCarveAst`, while source merges body boundaries and reports that flattening. A table foot now crosses through `ListTable's footer-rows`.
- **Column alignment now has a column-level spelling.** Positional `aligns` metadata preserves it even without a header row.
- **Citations come back as citations, without their definitions.** Quoted text, citations and block cells used to change AST shape on the way back and now return unchanged. What a citation still cannot carry is the bibliography - the bridge's own warning says entries live in pandoc metadata, so it emits no `[@key]`: definitions and an undefined key renders verbatim. The probe carries no bibliography, so that part is the bridge's account, not a measurement here.

Writer expressiveness, all 76 output formats

How many of the 66 features the writer can represent at all, broken out by group. Groups: 16 inline, 9 block, 10 list, 11 table, 15 attribute, 4 raw-passthrough probes. A raw score of 2/4 is normal and correct - a writer should emit raw content aimed at itself and drop raw content aimed at another target. The two highlighted rows are Carve through pandoc-carve, not pandoc built-ins.

WRITER	TOTAL / 66	INLINE / 16	BLOCK / 9	LIST / 10	TABLE / 11	ATTRS / 16	RAW / 4	READER?
json	66	16	9	10	11	16	4	-
native	66	16	9	10	11	16	4	yes
xml	66	16	9	10	11	16	4	-
carve-ast	65	16	9	9	11	16	4	yes
carve	64	16	9	9	10	16	4	yes
dzslides	63	16	9	9	11	16	2	-
html	63	16	9	9	11	16	2	yes
html4	63	16	9	9	11	16	2	-
html5	63	16	9	9	11	16	2	-
revealjs	63	16	9	9	11	16	2	-
s5	63	16	9	9	11	16	2	-
slideous	63	16	9	9	11	16	2	-
slidy	63	16	9	9	11	16	2	-
chunkedhtml	62	16	8	9	11	16	2	-
epub	62	15	9	9	11	16	2	yes
epub2	62	15	9	9	11	16	2	-
epub3	62	15	9	9	11	16	2	-
markdown	59	16	9	9	9	12	4	yes
commonmark_x	57	16	8	9	8	12	4	yes
commonmark	56	15	8	7	11	13	2	yes
djot	56	16	8	9	6	13	4	yes
beamer	55	16	8	10	9	10	2	-
jats	54	16	5	8	11	14	0	yes
jats_archiving	54	16	5	8	11	14	0	-
jats_publishing	54	16	5	8	11	14	0	-
latex	54	16	8	9	9	10	2	yes
docx	53	16	8	10	9	10	0	yes
typst	53	16	9	10	9	9	0	yes
jats_articleauthoring	52	16	5	6	11	14	0	-
markdown_strict	52	15	8	5	11	11	2	yes
context	51	16	7	9	10	9	0	-
mediawiki	51	15	8	8	9	9	2	yes
gfm	50	16	8	7	8	9	2	yes
markdown_mmd	50	16	8	5	8	9	4	yes

WRITER	TOTAL / 66	INLINE / 16	BLOCK / 9	LIST / 10	TABLE / 11	ATTRS / 16	RAW / 4	READER?
odt	49	15	8	10	9	7	0	yes
opendocument	49	15	8	10	9	7	0	-
asciidoc	48	16	8	8	9	7	0	yes
asciidoc_legacy	48	16	8	8	9	7	0	-
asciidoctor	48	16	8	8	9	7	0	-
markdown_phpextra	48	15	8	5	8	10	2	yes
markua	48	16	7	9	6	10	0	-
ipynb	47	16	8	5	8	8	2	yes
rst	47	15	8	9	8	4	3	yes
markdown_github	46	15	8	5	8	8	2	yes
textile	46	13	8	8	7	7	3	yes
muse	45	15	8	7	8	3	4	yes
docbook4	44	15	6	8	7	6	2	-
org	44	15	9	7	5	5	3	yes
docbook	43	15	6	8	7	6	1	yes
docbook5	43	15	6	8	7	6	1	-
icml	42	15	8	8	9	2	0	-
ms	41	15	8	8	7	3	0	-
pptx	40	15	6	8	9	2	0	yes
rtf	40	14	8	9	9	0	0	yes
tei	40	15	7	7	6	5	0	-
fb2	39	16	6	9	8	0	0	yes
ansi	38	13	8	8	7	2	0	-
dokuwiki	38	15	8	4	6	3	2	yes
texinfo	38	15	8	7	6	0	2	-
xwiki	38	13	8	4	4	7	2	-
jira	37	16	6	4	4	7	0	yes
t2t	37	15	7	4	6	2	3	yes
zimwiki	37	15	7	4	6	3	2	-
man	36	14	7	8	7	0	0	yes
haddock	35	13	6	7	9	0	0	yes
plain	35	10	5	9	9	2	0	-
bbcode_hubzilla	34	12	8	7	5	2	0	-
bbcode	33	11	8	7	5	2	0	-
vimdoc	33	8	6	9	9	1	0	-
bbcode_xenforo	32	12	8	4	5	3	0	-
bbcode_steam	30	12	8	4	5	1	0	-
bbcode_phpbb	29	11	8	7	2	1	0	-
bbcode_fluxbb	26	11	8	5	2	0	0	-
opml	1	0	1	0	0	0	0	yes
biblatex	0	0	0	0	0	0	0	-

WRITER	TOTAL / 66	INLINE / 16	BLOCK / 9	LIST / 10	TABLE / 11	ATTRS / 16	RAW / 4	READER?
bibtex	0	0	0	0	0	0	0	-
csljson	0	0	0	0	0	0	0	-
pdf	0	0	0	0	0	0	0	-

TABLE 2

What comes back, for the 34 formats pandoc also reads

The gap between the writer column and the exact column is the cost of using a format as storage rather than as output.

The last column lists features the writer emits correctly but the matching reader cannot recover.

FORMAT	WRITER / 66	ROUND-TRIP / 66	EXACT / 66	CANON. / 66	METADATA / 10	READER DROPS
native	66	66	66	-	10	-
carve-ast	65	64	65	-	10	list_loose
carve	64	63	64	-	10	list_loose
markdown	59	59	46	11	10	-
html	63	62	44	3	3.5	ol_example
commonmark_x	57	57	39	2	10	-
epub	62	61	36	2	2.5	ol_example
djot	56	56	35	-	0	-
rst	47	45	33	1	0	header_level, list_loose
muse	45	43	32	3	4.5	table_colspan, table_rowspan
markdown_mmd	50	50	28	3	6.5	-
markdown_phpextra	48	48	26	1	0	-
mediawiki	51	44	25	3	0	math_display, ol_style_roman, ol_style_alpha, ol_example, table_attrs, table_attrs_nocaption, table_row_attrs
typst	53	48	25	-	-	code_inline_attrs, list_loose, table_colwidth
gfm	50	50	24	2	10	-
latex	54	47	24	7	4	rawblock_tex, ol_start, ol_style_roman, ol_style_alpha, ol_delim_paren, list_loose, table_colwidth
docx	53	43	23	9	4	code_inline_attrs, softbreak, span_attrs, codeblock_lang, codeblock_attrs, div_attrs, ol_example, figure_attrs, table_attrs, table_attrs_nocaption
odt	49	38	23	3	0	smallcaps, code_inline_attrs, math_display, link_title, image_size, ol_start, ol_example, list_loose, table_align, table_cell_align, table_rowheadcols
docbook	43	41	22	5	4	math_display, div_attrs
markdown_github	46	46	22	3	0	-
asciidoc	48	47	21	-	2	header_level
commonmark	56	56	21	-	0	-
dokuwiki	38	35	21	1	0	rawblock_html, rawblock_tex, div_attrs
ipynb	47	66	21	3	0	-
jats	54	43	21	-	4	span_attrs, link_title, div_attrs, ol_start, ol_delim_paren, table_colwidth, table_attrs, table_attrs_nocaption, table_cell_attrs, table_row_attrs, table_rowheadcols
org	44	43	21	3	3.5	ol_delim_paren
textile	46	44	21	2	0	image_size, rawblock_tex
markdown_strict	52	52	18	-	0	-

FORMAT	WRITER/66	ROUND-TRIP/66	EXACT/66	CANON./66	METADATA/10	READER DROPS
jira	37	36	17	1	0	math_display
man	36	35	15	5	2	table_colwidth
rtf	40	26	15	1	0	code_inline, note, blockquote, codeblock, orderedlist, ol_start, ol_style_roman, ol_style_alpha, ol_delim_paren, list_loose, table_colspan, table_rowspan, table_align, tab
fb2	39	33	13	-	4	table_head, table_foot, table_colspan, table_rowspan, table_align, table_block_cell
haddock	35	32	13	1	0	list_loose, table_colspan, table_colwidth
t2t	37	35	13	-	0	rawblock_tex, div_attrs
opml	1	0	0	-	2	-
pptx	40	14	0	1	0	emph, strong, underline, strikeout, superscript, subscript, smallcaps, code_inline, math_display, linebreak, link, blockquote, codeblock, lineblock, bulletlist, orderedlist,

ipy nb's round-trip column is not meaningful: its *writer* stamps a fresh UUID on every cell and the reader hands it back as a Div id, so two conversions of the same document differ. Its exact column is measured after that wrapper is stripped and is trustworthy.

TABLE 3

Which elements are hard

Sorted by how rarely a feature makes it through a round-trip intact. Nothing is universal - opml's reader rejects almost every probe - but emphasis returns exactly in 32 of the 34 readers and bullet lists in 31, where example lists and cell alignment manage one each and citations two.

ELEMENT / FEATURE	GROUP	WRITERS/73	EXACT	ROUND-TRIP/33	CARVE	CARVE - AST
ol_example	list	25		0	◯	◯
table_cell_align	attrs	28		0	●	●
cite	block	16		1	●	●
figure_attrs	attrs	41		1	●	●
table_multibody	table	19		2	◯	●
table_row_attrs	attrs	19		2	●	●
table_rowheadcols	table	21		2	●	●
codeblock_attrs	attrs	25		2	●	●
table_attrs_nocaption	attrs	25		2	●	●
table_attrs	attrs	34		2	●	●
table_foot	table	66		2	●	●
rawblock_tex	raw	15		3	●	●
table_cell_attrs	attrs	19		3	●	●
code_inline_attrs	attrs	29		3	●	●
table_colwidth	table	43		3	●	●
span_attrs	attrs	45		4	●	●
div_attrs	attrs	46		4	●	●
table_rowspan	table	46		4	●	●
table_block_cell	table	66		4	●	●

ELEMENT / FEATURE	GROUP	WRITERS/73	EXACT	ROUND-TRIP/33	CARVE	CARVE-AST
header_attrs	attrs	19		5		
rawinline_html	raw	29		5		
link_attrs	attrs	35		5		
image_size	attrs	42		5		
table_colspan	table	47		5		
table_caption	table	65		5		
rawinline_tex	raw	10		6		
quoted_single	inline	68		6		
ol_style_roman	list	49		7		
lineblock	block	58		7		
quoted_double	inline	68		7		
rawblock_html	raw	32		8		
smallcaps	inline	50		8		
figure	block	64		8		
ol_style_alpha	list	51		9		
header_level	block	54		9		
ol_delim_paren	list	28		10		
ol_start	list	52		10		
table_align	table	54		10		
header	block	68		10		
orderedlist	list	68		10		
link_title	attrs	30		11		
image_title	attrs	46		11		
math_display	inline	54		13		
codeblock_lang	attrs	57		14		
underline	inline	66		14		
math_inline	inline	68		15		
table	table	68		16		
image	inline	68		17		
deflist	list	68		18		
table_head	table	66		19		
note	block	68		20		
subscript	inline	60		21		
superscript	inline	60		21		
strikeout	inline	67		21		
hrule	block	60		22		
softbreak	inline	48		23		
codeblock	block	68		24		
linebreak	inline	68		24		
code_inline	inline	64		28		
blockquote	block	66		28		

ELEMENT / FEATURE	GROUP	WRITERS/73	EXACT	ROUND-TRIP/33	CARVE	CARVE-AST
link	inline	66		28	●	●
list_nested	list	68		29	●	●
list_loose	list	42		39	●	●
bulletlist	list	68		39	●	●
emph	inline	66		31	●	●
strong	inline	66		31	●	●

TABLE 4

Document metadata

A standalone document carrying ten metadata values, written and read back. Filled dot = identical, half dot = present but reshaped, hollow = gone.

● exact ● partial ○ lost

META KEY	COMMONMARK_X	GFM	MARKDOWN	NATIVE	CARVE	CARVE-AST	MARKDOWN_MMD	MUSE	DOCBOOK	DOCX	FB2	JATS	LATEX	HTML	ORG	EPUB	ASCIIDOC	MAN	OPML	COMMONMARK	DJOT	DOKUWIKI	HADDOCK	IPYNB	JIRA	MARKDOWN_GITHUB	MARKDOWN_PHPEXTRA	MARKDOWN_STRICT	MEDIAWIKI	ODT	PPTX	RST	RTF	T2T	TEXTILE
title	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
subtitle	●	●	●	●	●	●	●	●	●	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
author	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
date	●	●	●	●	●	●	●	●	●	●	●	●	●	○	●	●	○	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
abstract	●	●	●	●	●	●	●	●	○	○	○	●	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
keywords	●	●	●	●	●	●	●	●	○	○	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
lang	●	●	●	●	●	●	●	●	○	○	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
custom	●	●	●	●	●	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
nested	●	●	●	●	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
flag	●	●	●	●	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

READING THE RESULT

Picking a target

Need everything preserved. json, native and xml are the AST itself and score 66/66 by construction. For anything that has to render, html5 and epub3 are the practical maximum.

Need it back later. carve through pandoc-carve leads at 64 exact (65 via its AST), then markdown 46 and commonmark_x 39. Metadata no longer separates them: markdown and Carve both return all ten meta values.

Print and typesetting. `latex` 54 and `beamer` 55 lead, `typst` 53 is close and loses mostly attributes, `context` 50 has the better table story but no raw passthrough at all.

Word processing. `docx` 53 is ahead of `odt` 47, and much further ahead on attributes (10 vs 5). `icml` keeps layout but almost no attributes.

Structured publishing. `jats` 54 is the strongest XML target and the only non-HTML writer with the complete table model; `docbook` and `tei` trail at 42 and 40.

Importing. The best sources are `html`, `docx`, `latex`, `jats`, `org`, `rst`, `mediawiki` and `markdown`. Avoid `pptx` and `rtf` as inputs when structure matters. Twelve input formats - `bits`, `creole`, `csv`, `endnotexml`, `mdoc`, `pod`, `ris`, `tikiwiki`, `tsv`, `twiki`, `vimwiki`, `xlsx` - have no matching writer and are not measured here.

Not document writers at all. `cs1json`, `bibtex` and `biblatex` serialize the references metadata and emit nothing for body content; `opml` keeps only the heading outline. Their low scores describe their purpose, not a defect.

LIMITS

What these numbers do not say

- **Expressed is not faithful.** `gfm` and `markdown_strict` "express" colspans and tables by falling back to a raw HTML block. That counts as a difference in the output bytes, and it is genuinely lossless, but the result is no longer that format's own markup.
- **SoftBreak is a flag artifact.** Under the default `--wrap=auto` almost every text writer folds `SoftBreak` into a space. With `--wrap=preserve` `html`, `markdown`, `latex`, `djot` and `typst` all distinguish it. Treat that one row as noise.
- **Dropping foreign raw is correct.** An `html` writer discarding `RawBlock` "latex" is target routing working as designed, not a coverage gap. Only formats that host both (`markdown`, `commonmark_x`, `markdown_mmd`, `muse`) reach 4/4.
- **Defaults only.** No extension tuning, no `--columns`, no custom templates. Turning on `markdown_strict+raw_html` or similar moves these numbers.
- **One feature at a time.** Interactions are untested - a format may keep an id on a heading and lose it on a heading inside a table cell.
- **Carve is measured through a bridge.** The `carve` and `carve-ast` rows come from `pandoc-carve`'s own converters, not from a `pandoc` reader/writer pair. Same probes, same two questions - but a library written against the `Pandoc AST` has an easier job than a writer constrained by an existing file format's syntax. Read the comparison as "what a user gets", not as a like-for-like implementation contest.
- **pdf scores zero because its engine is missing,** not because the format is poor. The writer shells out to an external `TeX` binary that is not installed here, so every pdf conversion errors and the row reads 0. Its real fidelity is `LaTeX`'s, minus whatever the engine rasterizes away.
- **Binary formats** (`docx`, `odt`, `pptx`, `epub`, `chunkedhtml`) are compared after unzipping and dropping the members that carry a timestamp - `core.xml`, `app.xml`, `meta.xml`. On the path the writer lane actually uses - `pandoc`'s server with `standalone` off - all 75 checkable writers re-emit byte-identical output for identical input, pdf excepted because it errors; `make_controls` reruns that check. Standalone output is not reproducible in the same way: a CLI `epub` embeds a random `urn:uuid` in `content.opf` and `toc.ncx`, and `ipynb` a random cell id.

How it was run

```
git clone https://github.com/markup-carve/pandoc-format-fidelity
cd pandoc-format-fidelity
make                # fetch pinned pandoc, run every lane, build the report
make carve CARVE_BRIDGE=../pandoc-carve/dist/index.js  # optional Carve lanes
```

The probe set lives in `src/probes.py`: each entry is a pair of pandoc AST block lists differing in exactly one feature. Writer conversions go through pandoc's own HTTP server for speed; the server has no filesystem access, so any media a probe references is handed over in the request; the CLI lanes get the same file through `--resource-path`. Both had to be fixed - an image pandoc cannot resolve makes epub leave the src alone and round-trip to a false *exact*.